

API SECURITY RISK ANALYSIS REPORT

1. Executive Summary

This report presents a security assessment of a public REST API, focusing on identifying potential vulnerabilities in authentication, authorization, and data exposure.

The analysis revealed **critical security gaps**, including lack of authentication and improper access control. While the tested API is designed for learning purposes, these issues highlight serious risks if present in production systems.

2. API Overview

Attribute	Details
API Name	JSONPlaceholder
API Type	REST API
Base URL	https://jsonplaceholder.typicode.com
Authentication	Not Implemented
Testing Scope	Read-only requests

Tested Endpoints:

- /users
- /users/1
- /posts
- /posts?userId=1
- /comments

3. Methodology

The security analysis was conducted using a structured and ethical approach.

Tools Used:

- Postman (API testing)
- Browser Developer Tools

Testing Approach:

- Sent GET requests to endpoints
- Analyzed responses and headers
- Checked authentication mechanisms
- Evaluated access control behavior

Scope:

- Read-only testing only
- No exploitation or system harm

4. Security Risk Analysis

Risk 1: Missing Authentication

Description:

All API endpoints are accessible without requiring authentication.

Evidence:

All requests returned 200 OK without any token or login credentials.

Business Impact:

- Unauthorized users can access sensitive data
- No accountability or user tracking

Severity:

High

Recommendation:

- Implement authentication (JWT / OAuth 2.0)
- Enforce token-based access

Risk 2: Broken Authorization (Insecure Direct Object Reference)

Description:

Users can access data of other users by modifying IDs in the URL.

Evidence:

Accessing /users/1 returns full user details without validation.

Business Impact:

- Exposure of other users' data
- Horizontal privilege escalation

Severity:

High

Recommendation:

- Implement Role-Based Access Control (RBAC)
- Validate user identity before granting access

Risk 3: Excessive Data Exposure

Description:

The API returns more data than necessary, including:

- Email addresses

- Phone numbers
- Full address details

Evidence:

/users endpoint exposes complete user profiles.

Business Impact:

- Privacy violations
- Increased risk of data scraping

Severity:

Medium

Recommendation:

- Return only required fields
- Implement response filtering

Risk 4: No Rate Limiting

Description:

The API allows unlimited requests without restriction.

Evidence:

Multiple repeated requests returned successful responses without delay.

Business Impact:

- Susceptible to abuse and DoS attacks
- Server performance degradation

Severity:

Medium

Recommendation:

- Implement rate limiting (e.g., 100 requests/minute)
- Apply throttling mechanisms

Risk 5: Weak Input Validation

Description:

Query parameters are accepted without strict validation.

Evidence:

Requests like /posts?userId=1 work without validation checks.

Business Impact:

- Potential injection vulnerabilities in real-world systems

Severity:

Low

Recommendation:

- Validate all inputs strictly
- Enforce proper data types

5. Risk Summary

Risk	Severity
Missing Authentication	High
Broken Authorization	High
Excessive Data Exposure	Medium
No Rate Limiting	Medium
Input Validation Issues	Low

6. Business Impact Analysis

If these vulnerabilities exist in a real SaaS environment:

- Sensitive customer data could be exposed
- Attackers could extract large datasets
- Unauthorized access may lead to breaches
- Increased risk of legal penalties (GDPR, privacy laws)
- Loss of user trust and brand reputation

7. Recommendations

To improve API security:

- Implement authentication (JWT / OAuth 2.0)
- Enforce Role-Based Access Control (RBAC)
- Limit response data (avoid overexposure)
- Apply rate limiting and throttling
- Validate all inputs strictly
- Monitor and log API activity

8. Conclusion

The API demonstrates multiple critical vulnerabilities, particularly in authentication and authorization mechanisms. While suitable for testing purposes, these weaknesses would pose significant risks in a production environment.

Addressing these issues is essential to ensure data protection, system reliability, and compliance with security standards.

9. Appendix

The image displays three screenshots of a REST client interface, likely Postman, showing API calls to the `https://jsonplaceholder.typicode.com` endpoint. The interface includes a sidebar with collections, environments, and specs, and a main area for the API call details.

Top Screenshot: The API call is `GET https://jsonplaceholder.typicode.com/posts`. The response is a 200 OK status, 51 ms, and 2.07 KB. The JSON body shows a list of posts:

```
1 {
2   {
3     "userId": 1,
4     "id": 1,
5     "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6     "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
7   },
8   {
9     "userId": 1,
10    "id": 2,
11    "title": "qui est esse",
12    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
13  },
14  {
15    "userId": 1,
16    "id": 3,
17    "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
18    "body": "et justo sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium
```

Middle Screenshot: The API call is `GET https://jsonplaceholder.typicode.com/users/1`. The response is a 200 OK status, 50 ms, and 1.4 KB. The JSON body shows a single user object:

```
1 {
2   "id": 1,
3   "name": "Leanne Graham",
4   "username": "Bret",
5   "email": "Sincere@april.biz",
6   "address": {
7     "street": "Kulas Light",
8     "suite": "Apt. 556",
9     "city": "Gwenborough",
10    "zipcode": "92998-3874",
11    "geo": {
12      "lat": "-37.3159",
13      "lng": "81.1496"
14    }
15  },
16  "phone": "1-770-736-8031 x56442",
17  "website": "hildegard.org",
18  "company": {
19    "name": "Romaguera-Crona",
20    "catchPhrase": "Multi-layered client-server neural-net",
```

Bottom Screenshot: The API call is `GET https://jsonplaceholder.typicode.com/comments`. The response is a 200 OK status, 86 ms, and 40.78 KB. The JSON body shows a list of comments:

```
1 [
2   {
3     "postId": 1,
4     "id": 1,
5     "name": "id labore ex et quam laborum",
6     "email": "Eliseo@gardner.biz",
7     "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
8   },
9   {
10    "postId": 1,
11    "id": 2,
12    "name": "quo vero reiciendis velit similique earum",
13    "email": "Jayne_Kuhic@sydney.com",
14    "body": "est natus enim nihil est dolore omnis voluptatem numquam\nnet omnis occaecati quod ullam at\nvoluptatem error expedita pariatur\nmnihil sint nostrum voluptatem reiciendis et"
15  },
16  {
17    "postId": 1,
18    "id": 3,
```



